Predator RF

Operator Guide

Pick this up cold. Zero to a working RF picture in under 30 minutes.

The single document. If the prior operator is unavailable, this is everything you need to pick the system up cold and conquer with it.

This guide is exhaustive on purpose. The minimum-kit and first-day workflows are at the top so you can be live in 30 minutes; the deep architectural sections at the back so you can run the full multi-node TOC, geolocate emitters with TDOA, push to ATAK, and run a clean mission lifecycle without prior context.

Table of contents

---

1. What this is, in one paragraph

Predator RF is a phone-first software-defined-radio cockpit for a single SIGINT operator. You plug a USB SDR into an Android phone (or a Linux box), point an antenna at the world, and the app shows you what's transmitting in real time — frequency, power, GPS-stamped location, and whether you've seen it before. It records baselines so the next sweep flags only what's new in the area, automates band scans with target/exclude lists, links to other phones or Raspberry Pi sensors as a private fleet (Kujhad), geolocates emitters by **time-difference-of-arrival (TDOA)** when you have ≥2 GPS-synchronized nodes, decodes P25 / RTL433 / POCSAG / ADS-B / AIS, and pushes selected hits to ATAK / TAK as Cursor-on-Target chat alerts. The whole system is **RX-only** — no transmit surface anywhere. There are explicit two-key gates on every output (CoT, AutoTasker re-tunes) so an automated assessment cannot bypass operator intent. Built on the SDR++ core. Sideloads as an APK; the Python backend ships as a systemd service.

---

2. Two deployment paths (you can run one or both)

Path 1 — Phone (Android)

The C++ Predator RF app on a Galaxy S22 (or any Android 10+ device), driving a USB-C SDR through an OTG cable. Standalone — no other infrastructure needed. The phone IS the cockpit. Optionally peers with other phones / RPi sensors via Kujhad over WiFi or a private overlay (ZeroTier / Tailscale).

Path 2 — Linux TOC + RPi sensor nodes (Python backend)

A Python backend service (`predator-rf.service`) runs on a Linux operator workstation or a Raspberry Pi. It:

- Owns persistence (SQLite mission ledger).
- Aggregates events from one or more Kujhad-equipped C++ sensor nodes (phones, RPi-with-SDR, etc.) via the Kujhad HTTP API.
- Runs the TDOA coordinator, decision engine, AutoTasker, and CoT emitter centrally.
- Exposes a REST + SSE API on `:8000` (token-protected) for any UI/dashboard or for other Predator-RF backends to chain through (CoC mode — Center of Control).

The two paths are designed to be **mixed**. A typical mission has one Linux workstation running the backend + ATAK plumbing, three phones in the field with the Android app each sharing their picture into Kujhad, and one or two RPi sensors dropped at fixed points for unattended overwatch. All of that fuses into one operator screen.

---

3. Minimum kit (you need ALL of these to do anything)

Item	Why
Android phone, Android 10 (API 28) or newer	Runs the app

USB-C OTG cable or adapter (USB-C → USB-A female)	Connects the SDR
A USB SDR — RTL-SDR Blog v4 ($40) is the cheapest, HackRF One ($340) is the upgrade	The actual radio
An antenna — even the rubber-duck shipped with the SDR works	Without one you receive nothing
The Predator RF APK	Built from the GitHub repo on Windows, or supplied by your team

That's it. No internet required after install. No subscription. No account.

A full tiered bill of materials (cheap → fleet) is in **§ 24**.

---

4. First-time install (sideload)

Predator RF is **not on the Play Store**. You sideload the APK.

1. On the phone: **Settings → About phone → tap "Build number" 7 times** until it says "Developer mode is ON".
2. **Settings → Developer options →** turn on **Install via USB** and **USB debugging**.
3. Get the APK onto the phone (USB cable from a computer, Google Drive, email — anything).
4. On the phone, open the file (Files app → Downloads → tap `app-debug.apk`).
5. When Android asks "Install unknown apps?" → **Allow** for whatever app you opened the APK from → **Install**.
6. Open **Predator RF**.

When the app first launches it will ask for these permissions — **grant all of them**:

- **Location** ("While using the app") → for the GPS-stamped hit map and TDOA participation
- **Storage / Files** → for saving baselines and exporting CSVs
- **USB device access** → pops up the first time you plug in your SDR; tap **Always allow for this device**

If you skip any of these the app still runs, but the matching feature goes dead-quiet (no map fix, no exports, no SDR).

---

5. Plug in the radio and start receiving

1. Connect your SDR to the phone with the USB-C OTG adapter. Antenna goes on the SDR.
2. The first plug-in pops a system dialog: **"Open Predator RF when this USB device is connected?"** — check the box and tap **OK**.
3. In the app, on the left rail tap the **source dropdown** (top-left, says `None` until a radio is selected) → pick `RTL-SDR`, `Airspy`, `HackRF`, etc. matching what you plugged in.

4. Tap ▶ **Start Listening** (big button under the source dropdown).
5. The waterfall on the right starts scrolling. **You're live.**

If the waterfall stays black, see § 23 (Troubleshooting).

---

6. The eight tabs (left-side rail, top to bottom)

The whole app is organized into eight tabs. They're labeled by their 3- or 4-letter code on the rail.

Code	Name	What it's for
SPEC	Spectrum	Live waterfall + tuner. Where you actually look at signals.
HITS	Hits & Events	Every signal the app has noticed, plus the running event log.
NET	Network	Catalog of known networks / talkgroups / aliases — your "rolodex of emitters."
MAP	Map	GPS-stamped hits on a map view, with TDOA fix dots, error ellipses, and node positions.
MIS	Mission Config	Mission mode (Manual / Classify / Scan / QuickScan), search bands, targets, excludes, dwell timing.
KUJ	Kujhad Fleet	Link this phone to other Predator RF phones / Raspberry Pi sensors over a network.
SYS	System	App settings — modules, themes, decoder bridges, ATAK/CoT, baseline comparison, TLS fingerprint.
BASE	Baseline	Record what the noise floor / normal traffic looks like, save it, then suppress it next time so only NEW signals fire as hits.

---

7. Mission modes (set on the MIS tab)

Mode	What it does	When to pick it
Manual	Direct operator tuning and marker ownership. Nothing automated.	Free recon, training, demonstration.

Classify	Keep manual control while idle resources watch the band — passive classification of whatever crosses the threshold.	When you're working a known signal but want background awareness.
Scan	Automated search and target workflow across all configured bands with full target/exclude logic.	The default field mode for a real op.
QuickScan	Rapid single-marker sweep for a quick check — no target persistence.	Walk into a new area, spend 60 s seeing what's hot, decide whether to stay.

Mode is set per-node. In a fleet, the operator workstation typically runs **Scan** while remote sensors run **Scan** with their own band lists tuned to their physical position.

---

8. The five day-one workflows

8.1 Just look around (passive recon)

1. **SPEC** tab → set the source → ▶ Start Listening.
2. Drag the waterfall left/right to retune. Pinch to zoom the view bandwidth.
3. Tap a peak to drop a marker on it.
4. To name what you just found: tap the marker → **Assign Marker** → tap **"Tap to edit"** in the popup → type a label.

8.2 Mark a hit and send it to your team

1. With a marker on a signal, tap **Assign Marker** if it isn't already.
2. The marker becomes a **hit** — visible on the **HITS** tab.
3. To push to ATAK / TAK: **SYS** tab → **ATAK / CoT** section → enter your TAK server IP + port → **E nable CoT**. The hit appears on the ATAK map as a friendly contact at your phone's current GPS location plus a chat message with the freq + power.

8.3 Record a baseline so you only see NEW signals

The first time you set up in a new area, run a baseline so the app learns what's "normally here" (paging towers, nearby commercial radios, broadcast leakage). Then any signal NOT in the baseline is flagged as new.

1. **BASE** tab → **Frequency Ranges** → set Range Name, Start Hz, Stop Hz → **+ Add Range**. Repeat for each band you care about. (Tap **"From Current View"** to grab whatever the waterfall is showing.)
2. **Recording** section → set a filename (or leave blank for auto-naming) → tap **▶ START RECORDING** (green).
3. Let it run for at least 5 minutes (longer = better — 30 min for a thorough sweep).
4. **■ STOP RECORDING** → **Save to File**.
5. **SYS** tab → **Baseline Comparison** → load the file → enable **Scan against baseline**. Set a threshold (default 6 dB). Now only signals exceeding the baseline by that margin become hits.

8.4 Run an automated scan across multiple bands

1. **MIS** tab → **Mission Mode** → **Scan**.
2. **Search Bands** section → add the bands you want swept (Name + Start Hz + Stop Hz). Tap **+ Add Band**.
3. **Targets** section — frequencies you specifically want to flag as priority (optional).
4. **Excludes** section — frequencies to ignore (your own gear, known broadcast carriers).
5. **Dwell** + **QuickScan Delay** + **Duration** — defaults are sane. Increase dwell for weak intermittent signals.
6. Back to **SPEC** → **▶ Start Listening**. The app sweeps the bands and drops hits on the **HITS** tab as it finds them.

8.5 Link a second phone or RPi sensor

See § 11 (full Kujhad walkthrough). Two-minute version:

- On the device-to-be-published-from: **KUJ** → **Listen** section → set port + Device name + API key → toggle **Listen** on.
- On the operator phone: **KUJ** → **Add Peer** → enter Name, Host (IP), Port, API key. Toggle **Mirr or peer spectrum** to view their waterfall.

---

9. Editing fields on a touchscreen

Every editable field works the same way:

1. Tap the field. A popup opens at the **top of the screen**, above the keyboard.
2. The keyboard slides up automatically.
3. Type. The popup is intentionally pinned high so the keyboard never covers it.
4. Tap **OK** (or hit Enter) to commit. **Cancel** to discard.

Letter input through some IMEs (composing keyboards) may not register inside the NativeActivity — if a popup goes blank when you type letters, switch the system keyboard to a non-composing

one (e.g. Google Keyboard with autocorrect off) for the duration of the edit. Numeric input always works.

---

10. The status bar at the top

Indicator	Color	Meaning
Thermal	Green / dim grey	Nominal
	Orange	SoC heating up; expect frame drops
	Red	Severe — back off (lower sample rate, get out of sun)
GPS	Green	Lock + age < 60 s
	Yellow	Lock but ageing — TDOA will refuse this node soon
	Red / dim	No lock — TDOA disabled, map fixes use last known position
Kujhad	Green	All peers connected
	Yellow	Some peers down
	Red	None reachable
CoT	Off / dim	Disabled
	Green	Enabled, packets going out
	Yellow	Enabled but stuck at the manual-approval queue

---

11. Kujhad fleet — protocol, pairing, TLS pinning

11.1 What it is

Kujhad is the in-band peer protocol. Each Predator RF instance can run as a **Device** (publishes its state and event stream) and/or a **Controller** (connects to one or more Devices and mirrors their state). It's hub-and-spoke, not mesh — the operator picks who they want to see.

11.2 The wire protocol (v1)

Tiny HTTP/1.1 + JSON, single API key in the `X-Kujhad-Key` header on every request. Default port **41947** (the C++ side; the Python backend's Kujhad client defaults to port **5259** in the env-var schema — use the actual port the Device is listening on).

Endpoint	Purpose
GET /v1/identify	Device name, version, role, hardware profile

GET /v1/gps	Current GPS fix
GET /v1/state	Mission mode, scan status, threshold, search bands, decoder roster
GET /v1/events?since=N	Hit / decoded-event stream since serial N
GET /v1/timing	Clock source (gpsdo/ntp/system), PPS lock, offset, drift, last-sync age
POST /v1/command	Issue a typed command — tune, scan, mission, identify

Every endpoint returns 401 on missing / wrong key, 400 / 404 / 405 on malformed / unknown / wrong-method.

Hard safety boundary: any command in the `tx` class is rejected by the dispatcher. The whole module never opens a transmit path.

11.3 Make this device discoverable (publish)

On the device that will publish its picture:

1. **KUJ** tab → **Listen** section.
2. **Listen port** — leave at default (41947) unless you have a conflict.
3. **Device name** — short identifier (e.g. `alpha-truck`, `bravo-roof`).
4. **API key** — leave the auto-generated 32-hex-char key, or paste your own. **Whatever this is, you'll need to enter the same value on every Controller that pairs with this Device.**
5. **Advertise address** — usually leave blank; the app picks the best interface (ZeroTier > Tailscale > LAN > loopback).
6. Toggle **Listen** ON.

The KUJ tab now shows a green "Listening on `<addr>:<port>`" banner. Hand the address + port + API key to whoever needs to pair.

11.4 Pair to a peer (subscribe)

On the Controller (your operator phone or workstation):

1. **KUJ** tab → **Add Peer**.
2. **Name** — anything memorable.
3. **Host** — the publisher's IP.
4. **Port** — the publisher's listen port.
5. **API key** — must match the publisher byte-for-byte.
6. **TLS** — leave OFF unless you've armed TLS on the publisher (see § 11.5).
7. **Add peer**.

The peer appears in the list with a status dot. Toggle **Mirror peer spectrum** to view that peer's waterfall on your screen while still controlling your local SDR. Toggle **Mirror peer markers** to see their hits on your map.

11.5 TLS pinning (when the build has OpenSSL)

By default the Kujhad protocol is plain HTTP, designed to ride on a private overlay (ZeroTier / Tailscale) where the network itself is the trust boundary. If your build links OpenSSL (`KUJHAD_HAVE_OPENSSL`), you can wrap the listener in TLS with a self-signed certificate that the controller pins by **SHA-256 fingerprint** — same model as SSH host keys.

On the publisher:

1. **SYS → Kujhad TLS → Generate self-signed cert** (creates `kujhad_tls_cert.pem` + `kujhad_tls_key.pem` in the app data dir, 10-year validity, RSA-2048).
2. The dialog shows the SHA-256 fingerprint as colon-separated hex. **Write this down** and read it to the controller operator out-of-band (voice, paper, SMS — NOT the same network you're about to pair on).
3. Toggle **TLS enabled**.

When TLS is on, plain HTTP is locked to loopback only — non-loopback peers attempting plain HTTP are rejected at the listener so the API key never crosses the wire in the clear.

On the controller:

1. **KUJ → Add Peer** → toggle **TLS** → paste the fingerprint into the **Pinned fingerprint** field.
2. On first connect the actual peer cert is compared against the pinned value. Mismatch = abort, with a loud warning.

11.6 Python backend → Kujhad device

To have the Linux backend (Path 2) consume from a C++ Kujhad device, set `FLEET_NODES` in `/etc/predator-rf/predator-rf.env`:

```
FLEET_NODES=alpha@192.168.1.10:41947:<api_key>:hackrf,bravo@192.168.1.11:41947:<api_key2>:r...
```

Format per node: `node_id@host:port:api_key:hardware_code`. The backend will identify each node, mirror its state, poll events at 1 Hz, GPS at 1 Hz, full state every 5 s, timing telemetry every 30 s. If the device reports a different hardware code than what's configured, the backend logs a loud warning and trusts the device — operators do mis-configure FLEET_NODES.

11.7 Kujhad troubleshooting

Symptom	Cause	Fix
Peer red / never green	Network unreachable, port blocked, or API key mismatch	Ping the host; telnet the port; verify the key character-for-character
Peer green but no events	Mission mode is Manual on the peer	Change to Scan / Classify on the peer
401 on every request	API key wrong	Re-copy the key

TLS handshake failure	Wrong fingerprint pinned, or peer cert was regenerated	Re-pin the new fingerprint after verifying out-of-band
Peer shows "hardware mismatch" warning	FLEET_NODES says rtlsdr but device reports hackrf	Update env; restart predator-rf; the backend always trusts the device

---

12. Geolocation — TDOA, the error ellipse, what confidence actually means

12.1 What TDOA is

Time-Difference-of-Arrival multilateration. When ≥2 GPS-synchronized sensor nodes hear the **same emission**, the difference in their timestamps (multiplied by the speed of light) gives a hyperbolic locus. With ≥3 nodes the loci intersect at a position fix.

12.2 What you need for a fix

Need	Why
≥2 distinct sensor nodes	Two events from the same receiver carry no time-difference info
GPS lock on each participating node	Without a position you can't triangulate at all
GPS lock age < 60 s on each node (configurable: GPS_MAX_AGE_S)	A stale fix means the node moved — the math breaks silently
Each node's hearings inside a 5-second window of each other	TDOA assumes one transmission; older measurements are pruned
A common time reference	GPSDO/PPS-disciplined hardware → high trust; system-clock-only → low trust but still produces a fix

The platform is **inclusive by policy**: any GPS-equipped node participates, even cheap RTL-SDRs without a GPSDO. Their timing trust just gets capped at 0.5 instead of the 0.98 a GPSDO+PPS HackRF gets, and the resulting fix's confidence is multiplied by that average. **A rough fix from 4 cheap nodes is still operationally useful** — it gives you a search area instead of nothing.

12.3 The math (for the operator who needs to argue with TOC)

- 2 distinct nodes → fall back to the midpoint of their positions, fixed `confidence = 0.3` (then scaled by timing trust).
- 3+ distinct nodes → iterative least-squares solve in a local ENU frame, 50 iterations;

geometric `confidence = min(0.95, 0.5 + 0.1·N)` where N is the measurement count, then scaled by timing trust.

- **Timing trust** per node:
  - GPSDO + PPS lock + |offset| < 10 ms → 0.98
  - GPSDO + PPS lock → 0.90
  - GPSDO no PPS → 0.75
  - NTP + |offset| < 25 ms + sync < 60 s → 0.70
  - NTP + |offset| < 100 ms → 0.55
  - NTP worse → 0.40
  - System-clock only → 0.30
  - Any of the above with last-sync > 5 min → minus 0.20
- **Final fix confidence** = geometric_confidence × mean(timing_trust of participants).

12.4 The error ellipse on the map

Every TDOA fix is rendered with a 1σ error ellipse, not just a dot. **This is the difference between a 50-metre fix and a 5-kilometre search area**, and both look like the same dot without it.

- **Base radius** scales as `50 m + (1 - confidence) × 4950 m` — a high-confidence fix shrinks toward 50 m, a zero-confidence fix grows toward 5 km.
- **Eccentricity** comes from the geometry of your participating nodes: tightly-clustered nodes give a near-circular blob, nodes strung along a line give a long thin ellipse perpendicular to the baseline (TDOA's actual physics — error is across the baseline, not along it).
- **Rotation** is the principal axis of the node cluster, rotated 90°, so the ellipse rotates with your fleet's actual geometry.

This is approximate (NOT Cramér-Rao-bound rigorous) but operationally correct. The operator immediately sees whether the system is confident in a position or merely confident there's some thing somewhere in a few-kilometre area.

12.5 What "confidence" actually means in three places

The word "confidence" appears on three different things — keep them distinct:

Where	What it is
track.confidence on the HITS row	Detection / classification confidence — how sure the system is that this is a real, persistent emitter (not a one-shot artifact). 0.1 at first sighting, climbs with corroborating observations.
track.location_confidence (drives the ellipse)	Geolocation confidence — TDOA fix quality only. Independent of the detection confidence.
assessment.confidence on the threat assessment	The intelligence layer's confidence in its threat assessment (not the detection or the location).

A track can have high detection confidence (0.9 — definitely real), low location confidence (0.2 — five-km search area), and medium assessment confidence (0.5 — probably elevated threat but not certain). All three are surfaced.

12.6 What happens with no GPS-synced nodes (single-phone operator)

If you have only one phone in the field, **TDOA does not run** — it physically can't, you need ≥2 GPS-synced sensors hearing the same emission within a 5 s window. Instead, the system has two fallback behaviours:

Default (RSSI proximity disabled — `RSSI_PROXIMITY_ENABLED=false`):

- The track's `estimated_lat / estimated_lon` stay `None`. Tracks have no map position of their own.
- The map shows your phone's GPS dot and its breadcrumb trail, with hit timestamps along the trail. **The operator does the triangulation in their head** by walking the area and noticing where the signal got stronger.
- This is the honest default: the system never invents an emitter position when it can't measure one.
- The CoT emitter (§ 16), if armed, falls back to **the detecting node's GPS** as a stand-in point so TAK has something to render — but that beacon means "I'm here and I heard something," not "the emitter is here." TAK shows it as a marker at your phone's location.

Opt-in (RSSI proximity enabled — `RSSI_PROXIMITY_ENABLED=true`): see § 12.7.

12.7 RSSI proximity fallback (single-node coarse geolocation)

When you set `RSSI_PROXIMITY_ENABLED=true`, single-node tracks get a coarse position estimate so the map shows something per emitter instead of just your phone's dot. **It is NOT a real geolocation. Treat the radius as a search area, not a position.**

How it works

- The detected signal's power (`power_dbfs`) is converted to absolute power at the antenna using a fixed offset (`RSSI_DBFS_TO_DBM_OFFSET`, default −30 dB).
- A **free-space path-loss** model converts that received power into a range estimate, given an **assumed transmitter EIRP** (`RSSI_ASSUMED_EIRP_DBM`, default 30 dBm = 1 W = typical handheld).
    - Formula: `d_m = (c / (4π·f_Hz)) · 10^((Pt_dBm − Pr_dBm) / 20)`
- The estimated range is multiplied by `RSSI_RADIUS_UNCERTAINTY_FACTOR` (default 2.0) to get the rendered circle radius — accounting for path-loss model error, EIRP guess error, and multipath.
- The radius is clamped to `[RSSI_MIN_RADIUS_M, RSSI_MAX_RADIUS_M]` (default 50 m to 5 km — same scale as a TDOA ellipse).
- The **circle is centred on the detecting node's GPS position**. There's no bearing information, so the emitter is "somewhere within radius `r` of your phone, in some unknown direction."
- `location_method = "rssi_proximity"` is set on the track so the UI can render it differently from a TDOA fix (a wide light circle vs. a tight ellipse).
- `location_confidence` is **hard-capped at 0.20** regardless of signal strength — TX power is unknown, so the system can never be highly confident about distance.

What this is good for

- **Walking-the-perimeter recon.** As you walk closer to a strong source, the estimated range shrinks and the circle visibly contracts on the map. You DF visually by watching the circles update.
- **Coarse "is it within 100 m or within 5 km" bucketing** when planning where to position fixed sensors.
- **Giving TAK a non-trivial CE radius** so the marker doesn't pretend to be a sub-metre fix when it isn't.

What this is NOT good for

- **Reporting actual emitter coordinates** to higher echelons. The single-phone CoT beacon should be understood as "operator was here, heard X" — not "emitter at coordinates Y."
- **Anything where the assumed TX power is unlikely to match.** A 25 W mobile being assumed-as-1 W will read as ~5× too close; a 100 mW IoT being assumed-as-1 W will read as ~3× too far. **If you know the band's typical TX power, set `RSSI_ASSUMED_EIRP_DBM` accordingly per mission.**
- **Positions in heavy clutter / urban canyon.** Free-space path loss assumes line-of-sight. Real-world buildings, trees, and ground bounce make the actual range estimate optimistic — bump `RSSI_RADIUS_UNCERTAINTY_FACTOR` to 3 or 4 in those environments.

Override priority

TDOA always wins. As soon as a second GPS-synced node hears the same emitter and TDOA produces a fix, the track's `location_method` flips to `"tdoa"` and the proximity circle is replaced by the proper ellipse. Operator manual-location overrides (§ 18.3) win over both.

What happens if even the detecting node has no GPS

- Track stays without `estimated_lat / estimated_lon`.
- Fallback CoT beacon, if armed, uses the **most-trustworthy node's last-known position** — marked with a different icon to flag that the location is a fallback.

---

13. Track lifecycle — NEW → TRACKING → STABLE → COASTING → LOST

Every emitter the system commits to becomes a **track** with a state machine:

State	Triggered by	Operator meaning
NEW	First detection	One sighting, not yet corroborated. Don't act on it alone.
TRACKING	≥3 observations	The system believes this is real. Scoring is live.
STABLE	≥10 observations	The emitter is well-characterised. AutoTasker may auto-task.

COASTING	Track aged out of last_seen window but within track_replay_window	"Was here, hasn't been heard in a while, expected to return."
LOST	Track aged out beyond replay window	Archived. Removed from the live picture.

The state is on the HITS row and gates how the rest of the system acts. AutoTasker and CoT escalation are both more conservative on NEW tracks.

---

14. Trust model — node trust score, timing trust, sensitivity trust

Every sensor node carries a composite **trust score** in `[0.05, 0.98]`. The score multiplies the weight of that node's observations during fusion, so a flaky cheap node doesn't drag a high-quality fix.

```
operational    = base_trust × uptime_fraction × (1 - false_positive_rate)
multi_node     = multi_node_agreement × 0.2
hardware       = freq_stability × 0.3
               + sensitivity   × 0.3
               + timing        × 0.2
               + 0.2 (constant)
trust_score    = (operational + multi_node) × hardware
                 × (0.7 if thermal_throttling else 1.0)
```

Component	What raises it	What lowers it
base_trust	Operator manually marks node as trusted	Default 0.6
uptime_fraction	Node reachable continuously	Network drops, restarts
false_positive_rate	Few unconfirmed-by-other-nodes hits	High solo-hit rate
multi_node_agreement	Hearings corroborated by ≥1 other node	Solo observations
freq_stability_trust	GPSDO-disciplined LO (low PPM)	Stock RTL-SDR tuner (50 PPM)

sensitivity_trust	Low NF (Airspy R2 at 2.5 dB)	High NF (HackRF at 10 dB)
timing_stability_trust	GPSDO + PPS, low offset	NTP only, large offset, stale sync
thermal_throttling	Cool device	Hot phone in sun → 0.7× multiplier

The hardware components are derived from the per-SDR capability table (§ 21).

---

15. The intelligence layer — anomaly flags → threat level → recommended act

Every track passes through `DecisionEngine.assess()` which combines anomaly flags + classification confidence + frequency-band context to produce an `AssessmentReport`:

Threat level	When it fires	Recommended action
unknown	No flags AND confidence < 0.3	continue_monitoring
low	At least one flag, low severity	continue_monitoring
medium	One high-severity flag OR ≥2 medium flags	increase_dwell_time
high	Two high-severity flags OR (one high + confidence ≥ 0.5)	focus_all_nodes (auto-tasks every TDOA-capable node to this freq)
critical	Any critical-severity flag	alert_operator_immediately (NEVER auto-actioned — operator pushes the button)

Tracks at `high` or `critical` set `escalate_to_atak = true`, which is one of the two gates the CoT emitter checks (§ 16).

The frequency-band context labels each track with the regulatory band — Aviation, VHF Public Safety, Marine VHF, UHF Public Safety, ISM 433/915/2.4 GHz, GNSS. The label flows into the assessment summary so the operator sees "Emitter at 162.5500 MHz (Marine VHF)" instead of just the raw frequency.

---

16. ATAK / TAK CoT integration — two-key gate, manual approval queue

16.1 The two-key gate

Predator RF starts in **RX-only** posture. Two flags arm CoT:

1. `cot_enabled` (env var `COT_ENABLED=true`, or the toggle in **SYS → ATAK / CoT**) — operator-level kill switch.
2. The track's most recent assessment must have `escalate_to_atak = true` (set automatically for `high` or `critical` threat levels).

Both must be true for a packet to leave. Even an automated `critical` assessment cannot bypass the operator's `cot_enabled` flag.

16.2 The manual approval queue (third key for the field)

In the field, set `COT_REQUIRE_MANUAL_APPROVAL=true` (or the **Require manual approval** toggle in SYS). With this on, `escalate_to_atak` no longer auto-fires. Each escalation enqueues at `GET /api/v1/approvals` (or surfaces as a notification on the operator phone), and the operator has to explicitly **Approve** before the packet goes out. **Reject** drops it; **Expire** happens after 2 hours by default (`COT_APPROVAL_EXPIRY_S`).

This is the two-person-rule equivalent for a solo operator. A single false-positive can spam TOC otherwise.

16.3 What goes on the wire

CoT 2.0 XML over UDP, defaults to multicast `239.2.3.1:6969` (the TAK SA feed). For unicast to a TAK Server, override `COT_DEST_HOST` and `COT_DEST_PORT`.

Field	What's in it
`type`	`a-u-G` (unknown ground unit) when there's a TDOA fix; `b-m-p-s-p-loc` (point of interest) when only a fallback location
`point lat / lon`	The TDOA fix, or the most-trustworthy node's GPS as fallback
`point ce` (circular error)	Scales 50 m (high confidence) → 5 km (zero confidence). TAK renders it as the circle around the marker.
`point hae / le`	9 999 999 (unknown altitude) — the platform doesn't claim altitude info
`contact callsign`	`<COT_UID_PREFIX>-<emitter_id_first_8_chars>`
`remarks`	`PREDATOR-RF <THREAT> | <freq> MHz | obs=<n> | conf=<x> | <summary>`
`stale`	Default 5 min after emit (`COT_STALE_S`), then TAK fades the marker
`__group name="Cyan" role="Team Member"`	Renders as a friendly contact, not a hostile

Per-emitter rate limit is **5 s** between beacons for the same emitter so a chatty source can't flood TOC.

16.4 Test the path before you go live

```
COT_ENABLED=true COT_DEST_HOST=<your TAK IP> COT_DEST_PORT=4242 \
  curl -X POST localhost:8000/api/v1/test/cot
```

The endpoint emits a synthetic beacon at your own GPS. If a marker appears in TAK within a few

seconds you're wired correctly.

---

17. AutoTasker — the action loop and its three brakes

When a `high` assessment recommends `focus_all_nodes`, AutoTasker re-tunes every TDOA-capable node to the track's primary frequency by issuing `POST /v1/command {class:"tune", action:"set", args:{frequencyHz, vfo}}` to each Kujhad device.

`critical` assessments are never auto-actioned — operator pushes the button.

Three brakes prevent runaway tasking:

1. **Per-node rate limit** — 30 s between tunes per node by default (`AUTO_TASKER_MIN_INTERVAL_S`). Prevents a chatty emitter from thrashing one node.
2. **Already-tuned check** — skip the tune if the node is already within ±2 kHz of the requested centre frequency.
3. **Global per-fleet budget** — at most 30 tunes/min across the entire fleet by default (`AUTO_TASKER_GLOBAL_MAX_PER_MIN`). Sized for ~6 nodes worth of churn. Prevents an assessment-loop bug from thrashing every node simultaneously at 0200 in the field.

AutoTasker is **OFF by default** (`AUTO_TASKER_ENABLED=false`). Same rationale as CoT — re-tune commands modify the SDR posture, so opt-in.

---

18. Operator overrides — friendly list, blacklist, manual location

Three classes of override, all persistent across restarts:

18.1 Friendly list

Mark an emitter_id as own-force or known-benign (your team's GMRS handhelds, the local police scanner, your own backhaul). Effect:

- Suppresses AutoTasker tunes for that emitter
- Suppresses CoT escalation
- Tags the track `friendly` on the UI (different icon, different colour)

Always recoverable — un-friend in the same UI.

18.2 Frequency blacklist

Add `(start_hz, end_hz)` ranges that the SweepCoordinator must skip and the TrackManager must drop on ingest. Use cases:

- A noisy commercial broadcaster you don't want clogging the HITS list
- A known interferer (your own LO leakage)
- An off-limits band you have a regulatory obligation NOT to log (e.g. some emergency-services frequencies)

18.3 Manual location override

Operator supplies a manual lat/lon for an emitter_id (confirmed via DF gear or visual). Wins over any TDOA estimate until cleared. Confidence is pinned to 0.95 by default. The map ellipse shrinks accordingly.

All three live in the mission DB and rehydrate on restart. There is an audit row for every add/remove so AAR exports show exactly when an operator declared something friendly or moved it manually.

---

19. Mission lifecycle — start, run, end, export the AAR

The mission ledger groups everything (events, tracks, assessments, approvals, overrides) under a `mission_id` so you can say "show me everything from yesterday's drill" or hand over a tarball as the after-action package.

19.1 Start

From the operator workstation:

```
TOKEN=$(grep API_BEARER_TOKEN /etc/predator-rf/predator-rf.env | cut -d= -f2)
curl -H "Authorization: Bearer $TOKEN" -X POST localhost:8000/api/v1/missions \
  -d '{"name":"OVERWATCH-20260315","operator":"K9-Actual"}'
```

Or from the phone: **MIS** tab → **Mission** section → **Start mission** → enter name → **Start**.

Starting a new mission auto-ends any in-flight one — no need to remember to close.

19.2 Active mission

```
curl -H "Authorization: Bearer $TOKEN" localhost:8000/api/v1/missions/active
```

19.3 End

```
curl -H "Authorization: Bearer $TOKEN" -X POST localhost:8000/api/v1/missions/end
```

19.4 Export the after-action package

```
curl -H "Authorization: Bearer $TOKEN" -OJ \
  localhost:8000/api/v1/missions/<mission_id>/export
```

Bundles a JSONL tarball: every event, every track, every assessment, every approval (approved AND rejected), every override change. Time-stamped and self-contained for ingest into another tool.

---

20. Path 2: the Python backend (TOC workstation + RPi sensors)

20.1 Install (one-time)

On a Linux box (Debian / Ubuntu / Raspberry Pi OS):

```
sudo mkdir -p /opt/predator-rf /etc/predator-rf /var/lib/predator-rf/backups
sudo git clone <repo> /opt/predator-rf
cd /opt/predator-rf
sudo python3 -m venv venv
sudo venv/bin/pip install -r requirements.txt
sudo cp deploy/predator-rf.env.example /etc/predator-rf/predator-rf.env
sudoedit /etc/predator-rf/predator-rf.env       # see § 20.3
sudo cp deploy/predator-rf.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable --now predator-rf
```

20.2 Where things live

Path	What
/opt/predator-rf	Source checkout + Python venv
/etc/predator-rf/predator-rf.env	All env-var config
/var/lib/predator-rf/mission.db	SQLite mission ledger
/var/lib/predator-rf/backups/	Snapshots from deploy/backup_mission.sh
/etc/systemd/system/predator-rf.service	systemd unit
journalctl -u predator-rf -f	Live log tail

Backend listens on `:8000`. For TLS, terminate at nginx / Caddy / Traefik in front — the backend itself stays plain HTTP intentionally so it works behind any proxy.

20.3 The full env-var reference

Every knob, defaults shown:

```
# API
API_HOST=0.0.0.0
API_PORT=8000
API_BEARER_TOKEN=               # empty = open (lab); set for any LAN deploy

# Fusion
TRACK_MAINTENANCE_S=10.0
TRACK_MERGE_S=30.0
MIN_CONFIDENCE=0.3

# Baseline learning
BASELINE_WINDOW_H=24.0
BASELINE_PRUNE_H=6.0

# Kujhad fleet (per-node spec: id@host:port:apikey:hardware)
FLEET_NODES=alpha@192.168.1.10:41947:KEY:hackrf,bravo@192.168.1.11:41947:KEY:rtlsdr

# TDOA
TDOA_ENABLED=true
GPS_MAX_AGE_S=60.0
TIMING_POLL_INTERVAL_S=30.0

# Persistence
PERSISTENCE_ENABLED=true
DATA_DIR=/var/lib/predator-rf
MISSION_DB=mission.db
TRACK_REPLAY_WINDOW_H=24.0

# CoT (RX-only by default)
COT_ENABLED=false
COT_DEST_HOST=239.2.3.1
COT_DEST_PORT=6969
COT_UID_PREFIX=PREDATOR
COT_STALE_S=300.0
COT_MULTICAST_TTL=1
COT_REQUIRE_MANUAL_APPROVAL=false  # SET TRUE IN THE FIELD
COT_APPROVAL_EXPIRY_S=7200.0
COT_APPROVAL_MAX_PENDING=200

# AutoTasker (RX-only by default)
AUTO_TASKER_ENABLED=false
AUTO_TASKER_MIN_INTERVAL_S=30.0
AUTO_TASKER_GLOBAL_MAX_PER_MIN=30

# RSSI proximity (single-node fallback geolocation; off by default)
RSSI_PROXIMITY_ENABLED=false
RSSI_ASSUMED_EIRP_DBM=30.0          # 1 W handheld; bump to 40 for 10 W mobile
RSSI_DBFS_TO_DBM_OFFSET=-30.0       # SDR-specific; calibrate per node if possible
RSSI_RADIUS_UNCERTAINTY_FACTOR=2.0  # 3-4 in cluttered environments
RSSI_MIN_RADIUS_M=50.0
RSSI_MAX_RADIUS_M=5000.0

# CoC mode (TOC-of-TOCs)
COC_MODE_ENABLED=false
```

```
COC_UPSTREAM_URLS=              # CSV: http://stationA:8000,http://stationB:8000
COC_RECONNECT_DELAY_S=5.0
COC_DEDUP_INTERVAL_S=15.0
COC_DEDUP_FREQ_TOL_HZ=5000.0
COC_DEDUP_LOC_TOL_M=500.0

# Observability
LOG_LEVEL=INFO
LOG_FORMAT=text                 # 'json' for ingest into Loki/Splunk/journald
METRICS_ENABLED=true
SHUTDOWN_DRAIN_TIMEOUT_S=5.0
```

20.4 Day-of operations (Linux side)

```
sudo systemctl restart predator-rf           # apply env changes
sudo systemctl status  predator-rf
journalctl -u predator-rf -f                 # tail
curl -s localhost:8000/healthz               # quick health
curl -s localhost:8000/metrics               # Prometheus-format
curl -s -H "Authorization: Bearer $TOKEN" localhost:8000/api/v1/nodes   # fleet
curl -s -H "Authorization: Bearer $TOKEN" localhost:8000/api/v1/tracks  # live tracks
curl -s -H "Authorization: Bearer $TOKEN" localhost:8000/api/v1/approvals  # pending
```

20.5 RPi sensor node

A Raspberry Pi running the Predator RF C++ build (or a slimmed sensor-only variant) acts as a Kujhad **Device**. Drop it at a fixed point with:

- An SDR (RTL-SDR Blog v4 minimum, Airspy or HackRF preferred for sensitivity)
- A GPS HAT or USB GPS (Adafruit Ultimate GPS HAT, BU-353N5)
- Power (PoE injector or 12 V → USB-C PD)
- Networking (Ethernet preferred; ZeroTier / Tailscale daemon installed for off-LAN reach)
- An antenna mounted as high as your mast allows

Configure its Kujhad **Listen** address + port + API key. Add it to FLEET_NODES on the operator workstation. Done — the operator now sees its picture.

20.6 CoC mode (Center of Control — TOC of TOCs)

Set `COC_MODE_ENABLED=true` + `COC_UPSTREAM_URLS=http://station-alpha:8000,http://station-bravo:8000` and this backend additionally consumes events from those upstream backends' `/api/v1/events/stream` SSE feed. Every aggregated event is tagged with `_upstream` so you know which station originated it, and `CrossStationDedup` coalesces tracks where freq + location agree (default tolerances: ±5 kHz, ±500 m, ±30 s co-occurrence). Two purely-local tracks never get merged here — that's `TrackAssociator`'s job.

20.7 RNS Layers (Reticulum transport for CoT)

The RNS layer is a **parallel** transport that pushes the same CoT/XML traffic Predator RF already sends to TAK over the existing UDP/TCP path and over Reticulum (RNS) at the same time — RNS handles its own path selection across whatever interfaces you've brought up. This is what gets

your CoT to TAK over LoRa, packet radio, I2P, or a long-range mesh when the regular IP transport is unavailable or untrusted.

The daemon ships in the Python backend (`backend/rns/`). It binds upstream RNS **1.2.0** exactly. Crypto stack: `cbor2` envelopes, Argon2id (t=3, m=64MiB, p=1) KDF, XChaCha20-Poly1305 IETF AEAD with the version byte bound as AAD (downgrade-resistant).

RX-side note: inbound CoT XML received over RNS is auto-forwarded to a local TAK app over UDP. On Linux it's opt-in (`RNS_ATAK_LOCAL_PORT` env var); on Android the port defaults to **4242** so peer-relayed CoT shows on the device's TAK map without operator action.

20.7.1 Where to access it

The RNS panel lives inside the Kujhad Fleet view of the Predator RF GUI on **both** Linux and Android:

Platform	How to open it	How it talks to the daemon
Linux (Kujhad GUI on the backend host)	Kujhad panel → "RNS Interfaces (Reticulum)" section	Unix socket (ControlServer), uid-checked, no network exposure
Android (Predator RF app, Tier 2+ deployment)	Same Kujhad panel — the C++ UI is rendered through NativeActivity so the layout is identical	android.net.LocalSocket against the same Unix-socket path

There is **no HTTP control plane**. The HTTP routes in `backend/api/routes/rns.py` exist as importable scaffolding but are deliberately not mounted in `backend/api/server.py` — the daemon control plane is local-only on every platform. If you need to script it, use the daemon's Unix socket directly or the helpers in `core/src/predator/kujhad_rns.h` (Linux) / `RnsBridge.kt` (Android).

The panel auto-refreshes at 1 Hz. Each row in the live status table shows the interface's `online` state, current RX/TX bytes, live bitrate, connected client count, and `last_error` if any.

20.7.2 The nine interface types — when to use each

Type	When to use it	Per-type required fields

tcp_client	Reach a remote RNS node by dialing its TCP listener (e.g. an RNS hub on the public internet, or another backend across an overlay)	target_host, target_port; optional kiss_framing, i2p_tunneled
tcp_server	Let other RNS nodes dial you on TCP (run a local RNS hub)	listen_port; optional listen_address, prefer_ipv6, i2p_tunneled
udp	Cheap stateless UDP transport, useful for LAN multicast-ish setups	listen_port; optional listen_address, forward_address, forward_port
i2p	Anonymized transport over I2P; the SAM bridge handles routing	optional peers, connectable, i2p_sam_address
auto_interface	Discover other Reticulum nodes on the same LAN automatically	group_id; optional discovery_scope (link/admin/site/organisation/global), discovery_port, data_port, allowed_interfaces, ignored_interfaces
rnode	LoRa via an RNode (the most common long-range field setup)	port (e.g. /dev/ttyUSB0), frequency_hz, bandwidth_hz, txpower_dbm (−10…30), spreadingfactor (7…12), codingrate (5…8); optional flow_control, id_callsign, id_interval_s
kiss_tnc	Generic packet-radio TNC running KISS	port, speed_baud; optional databits, parity, stopbits, preamble_ms, txtail_ms, persistence (0…255), slottime_ms, flow_control, beacon_interval_s, beacon_data
ax25_kiss	Amateur AX.25 packet radio (callsign-required)	everything kiss_tnc has plus callsign, ssid (0…15), axint_port

pipe	Wrap an arbitrary external process as an RNS interface (advanced)	command; optional respawn_delay_s

`reliable_cot` defaults by type (spec section C):

- **`rnode` → False** (LoRa airtime is precious; Link/Resource overhead defeats the point — packets are sent unconfirmed)
- **everything else → True** (TCP/UDP/I2P/Auto/Pipe/KISS variants get reliable mode by default — Link/Resource is cheap on those links, so CoT escalations are confirmed)

You can override per-interface in the UI.

20.7.3 Common fields (every interface, regardless of type)

Spec section B fields, applied by `_build_rns_interface` to every iface:

Field	Default	Purpose
id	auto (UUID4)	Stable identifier; never changes after creation.
name	required	Human label shown in the panel. Must be unique.
type	required	One of the nine types above.
enabled	true	Toggle without deleting. Disabled interfaces don't bind.
mode	full	full / gateway / access_point / roaming / boundary — RNS routing role. Use full unless you've read the RNS docs.
outgoing	true	Whether RNS may originate transmissions on this iface. RX-only nodes set false.
bitrate_hint_bps	unset	Hint for RNS scheduler; LoRa typically 1200, TCP 10^6, etc.
announce_interval_s	unset	How often this node announces itself on the iface.
notes	unset	Free-form operator note shown in the panel; not transmitted.
reliable_cot	type-default	Per-iface override of the publish reliability mode.
ifac_netname	unset	Reticulum Interface Access Code network name. Pre-shared identifier that scopes which nodes can talk on this iface — non-keyed nodes see only undecodable framing. Must be set together with ifac_netkey.
ifac_netkey	unset	Pre-shared passphrase that derives the IFAC keyed-hash material. Treat as a secret — minted into replication tokens encrypted under the operator passphrase, never stored or logged in cleartext.

ifac_size	unset	Truncation length in bytes of the IFAC keyed hash, range 8..512. Leave unset to use Reticulum's internal default. Larger = more spoofing-resistant, more per-frame overhead.

20.7.3.1 IFAC — when to use it

IFAC is Reticulum's per-interface pre-shared-key gate. With both `ifac_netname` and `ifac_netkey` set, every Reticulum frame on that interface is hashed with the netkey so nodes that don't share the key can't decode link-layer framing at all — your traffic is invisible to them, not just rejected.

Use IFAC when:

- You're sharing a physical link layer with other Reticulum users (e.g. an open LoRa frequency at a hamfest, a public TCP hub) and want operational separation rather than just allowlist filtering.
- You want the link layer itself to refuse non-mission traffic before any RNS Transport / Identity / allowlist logic runs (defence in depth — IFAC is layer-1.5, the peer allowlist is layer-3).
- You're running an `auto_interface` on a multi-tenant LAN and want only your team's nodes to discover each other.

Don't use IFAC when:

- You're on a dedicated point-to-point link (TCP client to your own hub, AX.25 to a single peer) — the peer allowlist already gates trust and IFAC adds per-frame overhead with no extra security gain.
- You're operating under amateur radio rules that forbid encryption — IFAC keyed framing **is** a form of obscurement and may not be permitted on Part 97 frequencies. Check your local regs.

Both fields are required for IFAC to take effect — setting only `ifac_netname` does nothing. The UI emits the IFAC block to the daemon only when both fields are non-empty (the C++ Save handler in `core/src/gui/main_window.cpp` checks `if (eIfacNetname[0] && eIfacNetkey[0])`); the daemon's `_build_rns_interface` checks the same condition before applying any IFAC attribute to the iface object.

IFAC fields ride through replication tokens. Mint a token from one node and the IFAC netname + netkey are bundled inside the AEAD-encrypted payload alongside the rest of the config — no separate out-of-band coordination required. If you'd rather have the receiving operator set their own IFAC (e.g. you're sharing a config but each operator has a different netkey policy), clear the IFAC fields in the UI before clicking **Mint token**.

20.7.4 Adding an interface — UI flow

1. Open the Kujhad panel → "RNS Interfaces (Reticulum)".
2. Click **Add interface**.
3. Pick a `type` from the combo. The form below regenerates with the COMMON section + the per-type fields for the chosen type.
4. Fill in the required fields (the form validates inline before letting you save — same `validate_interface` function the daemon uses, so a UI-accepted config will not be rejected by the

daemon).
5. Click **Save**. The daemon writes the new entry to its config and brings the interface up. Watch the live status row for `online=true`. If `last_error` populates, fix the field and click **Restart** on the row.

20.7.5 Restart behavior

When you click **Restart** (or call `restart_interface`), the daemon:

1. **Drains** the current iface for `drain_timeout_s` (default 5 s). Pending packets either flush or are dropped.
2. **Spawns** the new iface from the current config.
3. **Waits up to `start_timeout_s`** for the iface to report up.
4. Returns `{forced_close, timed_out, last_error}`. A forced teardown (a hung iface that wouldn't drain) surfaces as `last_error="forced"` per spec section G.
5. Per-interface peer entries are purged on teardown (rebuilt as remote nodes re-announce).

Restart all does the same in sequence across every iface.

20.7.6 Replication tokens — moving a config between nodes

A replication token is a passphrase-encrypted bundle of the daemon's config (and optionally its identity) that you can mint on one node and import on another to clone the RNS posture without retyping every field.

Mint (Kujhad → Mint token):

1. Click **Mint token** in the panel.
2. Choose `include_identity` (yes = the importer becomes "you" on RNS — same node hash; no = importer keeps its own identity).
3. Enter a strong passphrase. Argon2id derives the AEAD key.
4. Copy the resulting `prf-rns-v1.*` string. Share it out-of-band.

Device-local placeholders: fields marked `DEVICE_LOCAL_FIELDS` in `schema.py` (e.g. `tcp_server.listen_address`, `udp.listen_address`, `i2p.i2p_sam_address`, `rnode.port`, `kiss_tnc.port`, `ax25_kiss.{port,axint_port}`, `auto_interface.{allowed_interfaces,ignored_interfaces}`) are replaced with `{"$placeholder": "<field_path>"}` markers during mint. The importer is re-prompted for them. **This is how you avoid leaking `/dev/ttyUSB0` paths or LAN IPs across an op.**

Import (Kujhad → Import token):

1. Click **Import token**, paste the string, enter the passphrase.
2. The importer prompts for every device-local placeholder. Provide the local-to-this-machine value for each (`/dev/ttyUSB1`, `192.168.5.10`, etc.).
3. Daemon validates the resulting config; refuses to apply if any field is invalid.
4. If `include_identity` was set at mint time, the importer also writes `identity.prv` (mode 0600), reloads `RNS.Identity`, rebuilds the IN destination, and re-syncs `bridge.own_hash16`. The node hash is preserved.

Scripted import (Linux): `deploy/rns-setup.sh` accepts a token + a `PRF_RNS_PLACEHOLDERS_J`

SON env var (or prompts on /dev/tty, retrying up to 5 times). --non-interactive fast-fails with the missing list. Env var encoding: __ → . (so interfaces__0__listen_address=192.168.5.10 maps to interfaces.0.listen_address).

20.7.7 Peer allowlist — gating who can announce in

peer_allowlist (a list of identity hashes in the daemon config) restricts which announces the bridge accepts. An announce from an identity not in the allowlist is dropped at the announce handler — that peer's OUT destination is never built, so envelopes are never fanned to it. Leave it empty to accept any peer that announces on predatorrf.cot.v1 (default — fine for closed deployments). Populate it for higher-trust ops.

The allowlist is **synced from the daemon config to the bridge at startup** (backend/main.py line 103). Changes via UI take effect on the next config reload.

20.7.8 Status fields (what the panel shows live)

status() returns:

Field	Meaning
daemon	running / stub (the latter when the rns Python package isn't importable)
identity_hash16	This node's RNS hash (16 hex chars)
interfaces[]	Per-iface live: online, rxb (bytes), txb, bitrate (current measured), clients (remote count), last_error, forced_close, timed_out, ifac_active (bool), ifac_netname (echoed back when active so the UI can render a tooltip; netkey is never echoed in status)

In the Kujhad live-status table this surfaces as a dedicated **IFAC** column — [IFAC] in green when the daemon reports ifac_active=true, dash otherwise. Hovering the badge shows the IFAC netname tooltip; the netkey itself is not sent back over the control plane and so is never visible in the UI after the initial save. | cot_bridge | Stats from the bridge: published, received, dropped_dedupe, dropped_loop, dropped_allowlist, dropped_invalid | | peer_allowlist_size | Count of allowlisted peers |

20.7.9 Logs

get_logs(level, since_seq) returns from the daemon's in-memory log buffer (last N entries). The Kujhad panel renders this as a tail at the bottom — filter by level (DEBUG/INFO/WARN/ERROR).

20.7.10 Identity & state files (Linux)

Path	Purpose
/var/lib/predator-rf/rns/identity.prv	RNS identity private key (mode 0600)

/var/lib/predator-rf/rns/config.json	Daemon config — interfaces[], cot_bridge, peer_allowlist, schema_version
/var/lib/predator-rf/rns/control.sock	Unix control socket (uid-checked)
/var/lib/predator-rf/rns/logs/	Rotating daemon logs

Backup the identity file before any major change. Losing `identity.prv` means losing your node's hash — every peer that allowlisted you needs to re-add the new hash.

20.7.11 Common field workflows

A. Bring up a 2-node LoRa mesh between two RNodes (915 MHz ISM)

- Both nodes: add an `rnode` iface — `port=/dev/ttyUSB0`, `frequency_hz=915000000`, `bandwidth_hz=125000`, `txpower_dbm=17`, `spreadingfactor=8`, `codingrate=5`, `id_callsign=YOURCALL` if licensed.
- Wait ~1 announce interval. Each panel's status table should show the other node under `clients`.
- CoT publishes from either node's backend reach the other's TAK app over LoRa (with `reliable_cot=false` — single-pass, unconfirmed).

B. Bridge to a remote RNS hub over the public internet

- Local: `tcp_client` → `target_host=hub.example.com`, `target_port=4242`, `mode=full`.
- Hub: typically already running `tcp_server`. Mint a replication token without identity, import on local with the hub's address as the placeholder.

C. AX.25 packet-radio gateway with a real callsign

- `ax25_kiss` iface — `port=/dev/ttyUSB0`, `speed_baud=9600`, `callsign=YOURCALL`, `ssid=7`, `axint_port=ax0` (the kernel AX.25 interface name).
- Set `id_callsign` if you want periodic `ID` beacons.
- `reliable_cot=true` makes sense here (AX.25 is slow but reliable enough that Link/Resource works).

D. RX-only listening node

- Set `outgoing=false` on every iface. The node will receive announces and inbound CoT but will not originate transmissions.

20.7.12 Security posture

Threat	Mitigation
Remote attacker reaches the daemon control plane	Not possible. The control plane is a Unix socket; HTTP routes are not mounted. Linux uses uid checks; Android uses LocalSocket (kernel-enforced same-app boundary).
Replication token sniffed on the wire	Argon2id+XChaCha20-Poly1305-IETF; AAD-bound version byte blocks downgrade. The token is useless without the passphrase.

Importing a leaked token	Use a strong passphrase; mint with include_identity=false if you don't trust the recipient with your node hash.
Hostile peer floods the bridge	peer_allowlist keeps fanout O(allowlisted peers). Per-peer LRU dedup (4096 entries / peer) drops repeated CoT.
LoRa replay / loop	Bridge-level loop suppression on own src tag; per-peer dedupe LRU.
RNS interface hangs and blocks shutdown	Restart drains for drain_timeout_s, then forces close — last_error="forced" shows in the panel.
TX-class command injected via Kujhad	Dispatcher unconditionally rejects any tx-class command; the module never opens a transmit path beyond what you configure on RNS interfaces directly.

20.7.13 Troubleshooting

Symptom	Cause / fix
daemon=stub in status	rns Python package not installed. pip install -r backend/rns/requirements.txt and restart.
Iface online=false, last_error="..."	Read the error string. Most common: serial port permissions (sudo usermod -aG dialout $USER), wrong port path, port already open by another process.
LoRa peers can hear you but you never see them	Check announce_interval_s on both sides. Default unset means RNS uses its built-in heuristic; set to 600 (10 min) for a stable mesh.
Iface online=true but you and a peer can't see each other and the link looks healthy	One side has IFAC set and the other doesn't, or the ifac_netname / ifac_netkey differ. With IFAC mismatched the framing is undecodable so the other node looks completely silent — not "rejected", just invisible. Either clear IFAC on both sides or copy the same netname + netkey to both.
You enabled IFAC and now your own restart makes the iface look dead	ifac_size outside 8..512 is rejected by validate_interface; if you typed a value the schema accepted but RNS doesn't like, check the daemon log for an attribute error. Fall back: clear ifac_size (=0) so RNS uses its default.
cot_bridge.dropped_allowlist > 0	A peer is announcing but not in your allowlist. Either add their hash or accept that you're filtering them out.
Inbound CoT not appearing in TAK on Android	Check RNS_ATAK_LOCAL_PORT (defaults to 4242 on Android, opt-in elsewhere). Make sure ATAK is listening on that UDP port.
restart_interface returns forced_close=true, timed_out=true	The iface hung in shutdown. RNS occasionally does this on serial USB resets. Re-plug the radio, click Restart again.
Token import re-prompts for fields you don't recognise	Those are device-local placeholders from the source config. Provide the appropriate value for your machine (e.g. your serial port, your LAN IP).

Identity hash changed after import_config	Token was minted with include_identity=true. By design — you now share that node's identity. If you wanted to keep your own, re-import a token minted without identity.

21. Hardware capability table (every supported SDR)

These drive the trust calculus. Pick your hardware knowing what trust score you'll get.

SDR	Freq range	Max sample	NF	MDS	TDOA-cap	Timing uncertainty	Price
RTL-SDR Blog v4	25 MHz–1.7 GHz	3.2 MS/s	6.0 dB	-110 dBm	NO	1000 ns	$40
HackRF One	1 MHz–6 GHz	20 MS/s	10.0 dB	-100 dBm	YES	500 ns	$300
Airspy R2	24 MHz–1.7 GHz	20 MS/s	2.5 dB	-125 dBm	YES	50 ns	$170
LimeSDR-USB	100 kHz–3.8 GHz	61.4 MS/s	3.0 dB	-120 dBm	YES (PPS out)	100 ns	$600
bladeRF 2.0	47 MHz–6 GHz	61.4 MS/s	4.0 dB	-118 dBm	YES	80 ns	$480
ADALM-PLUTO	325 MHz–3.8 GHz	61.4 MS/s	5.0 dB	-115 dBm	YES	200 ns	$200
SoapySDR generic	1 MHz–6 GHz	10 MS/s	8.0 dB	-105 dBm	NO	500 ns	varies

Practical guidance:

- **For TDOA you want Airspy or LimeSDR.** Both have low timing uncertainty AND great sensitivity. LimeSDR's PPS output lets you discipline the LO from a GPSDO directly.
- **HackRF is the workhorse for HF coverage** — only SDR in the list that goes below 24 MHz. Acceptable for TDOA; not the best.
- **RTL-SDR Blog v4 is the cheap "I'm here too" node.** No TDOA timing path, so it gets the 0.5-cap timing trust. Still useful as a 4th observer to pull a fix's confidence up.
- **PlutoSDR is the budget TDOA node** — half the price of an Airspy with TDOA capability.

22. Field-day checklist (print this)

Pre-departure (in shop / vehicle, with WAN)

- [ ] Phone(s) charged + power bank packed
- [ ] APK version verified — open app → SYS → check version against your team's current build
- [ ] SDR + antenna + USB-C OTG cable in the bag (one set per phone)
- [ ] At least one **baseline file** for the area you're going to (record ahead if possible)
- [ ] If using ATAK: TAK server reachable + credentials entered + a successful test beacon yesterday
- [ ] If using Kujhad fleet: peers added, all on the same overlay (ZeroTier / Tailscale / LAN), API keys match, TLS fingerprints pinned if TLS is on
- [ ] If running the Linux backend: `sudo apt update && sudo apt upgrade -y` on workstation + each RPi
- [ ] Mission DB backed up: `deploy/backup_mission.sh` to USB
- [ ] System time disciplined: `chronyc tracking` shows `Leap status : Normal`
- [ ] `/etc/predator-rf/predator-rf.env` reviewed; `FLEET_NODES` matches today's node serials
- [ ] `API_BEARER_TOKEN` rotated for this mission: `openssl rand -hex 32`
- [ ] CoT/TAK destination + UID prefix set ONLY if you intend to push to TOC
- [ ] `COT_REQUIRE_MANUAL_APPROVAL=true` if `COT_ENABLED=true` (two-key gate)
- [ ] `AUTO_TASKER_ENABLED` matches your ROE — leave OFF unless you're authorized to re-tune nodes

On-site, before fleet power-on

- [ ] Each sensor node placed; antennas oriented; GPS sky-view confirmed
- [ ] Power budget sanity-checked (battery / vehicle alternator vs. node draw)
- [ ] Network reachable: `ping <node-ip>` from operator workstation
- [ ] Each node's clock disciplined (GPSDO PPS lock visible, or `chronyc tracking` green)

Bring-up sequence

1. Power on all sensor nodes; wait 60 s for GPS lock + Kujhad ready
2. Operator workstation: `sudo systemctl start predator-rf`
3. `python deploy/preflight.py` → must report **GO**
4. `journalctl -u predator-rf -f` → no ERROR lines in the first 30 s
5. `curl http://localhost:8000/healthz` → `"status":"ok"`
6. `curl -H "Authorization: Bearer $TOKEN" localhost:8000/api/v1/nodes` → every expected node listed, `gps_synchronized=true` on TDOA-capable ones
7. `curl -H "Authorization: Bearer $TOKEN" -X POST localhost:8000/api/v1/missions -d '{"name":"<callsign-YYYYMMDD>"}'`
8. On the operator phone: SPEC → ▶ Start. Verify waterfall + GPS green + KUJ green + CoT (if armed) green.

In-mission checks (every 30 minutes)

- [ ] Glance at thermal indicator on each phone
- [ ] Glance at GPS lock on each phone
- [ ] `curl /metrics` → `predator_events_total` is climbing

- [ ] `curl /api/v1/android-pull?since_ns=0` → all nodes show `gps_lock=true` AND `gps_age_s < 60`
- [ ] No CoT approvals stuck > 5 min in `/api/v1/approvals` (operator backlog)
- [ ] Phone batteries > 20% — swap to power bank before 10%

End of mission

1. Phones: MIS → mode back to Manual; HITS → export hits to CSV; BASE → save fresh baseline if recorded
2. Phones: ▶ Stop Listening → close app → unplug SDR
3. Workstation: `POST /api/v1/missions/end`
4. `GET /api/v1/missions/<id>/export` → save the AAR tarball
5. `deploy/backup_mission.sh` → USB

---

23. Troubleshooting (every symptom we've actually seen)

Symptom	Most likely cause	Fix
Waterfall is black after Start Listening	SDR not selected, or USB permission denied	Source dropdown → re-pick. Unplug + replug SDR → "Always allow" on dialog.
App says "device busy"	Another app (or previous Predator RF session) holds the SDR	Force-stop in Settings → Apps; unplug + replug
Touch feels unresponsive / wrong widget fires	Old build — fixed in the touch-passthrough + ID-stack patches	Update APK
Soft keyboard covers the input field	Old build — fixed by IME-inset clamp	Update APK
GPS never locks	Phone needs sky view; indoor/urban canyon = no fix	Step outside; wait 60 s
Letters don't appear when typing in popup	NativeActivity IME composing path	Switch to non-composing keyboard for the edit; numeric input always works

Kujhad peer red / disconnected	Network unreachable, port blocked, API keys differ	Ping the peer from a terminal app; verify keys character-for-character
Kujhad TLS handshake fails	Wrong fingerprint pinned, or cert was regenerated	Re-pin the new fingerprint (verify out-of-band first)
Kujhad peer green but no events	Peer is in Manual mission mode	Switch peer to Scan or Classify
ATAK marker never appears	Wrong server IP/port, or filtering by UID prefix on TAK server	Check TAK server logs; test with curl -X POST /api/v1/test/cot
ATAK markers stop coming	Approval queue is on and the operator hasn't approved	Check GET /api/v1/approvals; approve or disable manual approval
Phone hot, framerate drops	Thermal throttling	Get out of sun; reduce sample rate (SPEC → source settings); trust score drops 30% while throttled
App crashes on startup after install	APK was sideloaded over a different signing key	Uninstall completely (Settings → Apps → Predator RF → Uninstall), reinstall
TDOA fix never appears on map	< 2 GPS-synced nodes hearing the same emission within 5 s	Check /api/v1/nodes — at least 2 must show gps_synchronized=true and gps_age_s < 60
TDOA ellipse is huge (5 km)	Low-confidence fix — only 2 nodes, or all timing-trust-capped RTL-SDRs	Add more nodes; deploy at least one Airspy or HackRF with GPSDO
TDOA fix wildly off	One node's GPS is stale	GET /api/v1/nodes and check gps_age_s per node; the offending one will be > 60 s

AutoTasker not retuning	AUTO_TASKER_ENABLED=false, OR the global-budget brake is firing	Check env, then /metrics for predator_autotasker_* counters
AutoTasker retuning the wrong nodes	DecisionEngine recommends only TDOA-capable nodes for high threats	This is by design; mark known-good non-TDOA nodes as friendly to suppress
Backend won't start: ERROR: address in use	Another process owns :8000	lsof -i :8000; kill or change API_PORT
Mission ledger growing huge	No prune	deploy/backup_mission.sh to archive; truncate per your retention policy
401 on every API call	API_BEARER_TOKEN set but no header sent	Add -H "Authorization: Bearer $TOKEN"

---

24. Bill of materials — pick your tier

Prices USD, May 2026, ballpark.

Tier 0 — Bare minimum (~$48)

- Android phone you already own — $0
- RTL-SDR Blog v4 — $40
- USB-C OTG adapter — $8
- Rubber-duck antenna (included) — $0

What you can do: live spectrum, hits, baseline, scans up to 1.7 GHz on a single phone. No HF, no fleet, no TDOA.

Tier 1 — Solo field operator (~$330–540)

- Tier 0 kit — $48
- Airspy Mini ($130) **or** HackRF One ($340)
- Diamond RH-77CA telescoping whip — $50
- External USB GPS (BU-353N5) — $35
- Powered USB-C OTG hub — $25
- 20,000 mAh USB-C PD power bank — $40

What you can do: above plus HF (HackRF), all-day battery, fast GPS, ATAK-ready.

Tier 2 — Solo TOC + one remote sensor (~$1,400)

- Tier 1 kit (HackRF flavor) — $540
- Raspberry Pi 5 8 GB + case + cooler + PSU — $130
- 256 GB A2 microSD — $25
- Second RTL-SDR Blog v4 (for the Pi) — $40
- GPS HAT (Adafruit Ultimate GPS HAT) — $45
- Linux laptop you already own OR add $500 for one
- Outdoor antenna mount + 25 ft LMR-400 + N-to-SMA pigtails — $80
- Diamond D130J discone — $130
- Pelican 1450 case — $130
- ZeroTier / Tailscale (free tier) — $0

What you can do: drop-and-walk sensor node, distributed collection, mission ledger, after-action exports, ONE TDOA pair (phone + RPi).

Tier 3 — Small team / multi-node fleet ($5,000–$10,000+)

- Tier 2 kit — $1,400
- 3 more RPi sensor nodes (~$500 each) — $1,500
- At least one **Airspy R2 + GPSDO** for high-trust TDOA — $200
- Cellular hotspot + data plan — $300
- OR mesh radio link (goTenna Pro X2 / RAJANT) — $500–$5,000
- Commercial directional antennas (Yagi for DF, dipoles per band) — $400
- 10–25 ft fiberglass tactical mast — $150
- Pelican / Storm cases per node — $400
- Operator laptop (rugged ThinkPad / Dell) — $500–$2,000

What you can do: full-perimeter overwatch from one operator screen, real TDOA fixes (multiple GPSDO-disciplined nodes, tight ellipses), ATAK to higher-echelon TOC, ledger across the fleet, real DF capability.

---

25. Glossary

Term	Meaning
AAR	After-Action Report — the mission ledger export tarball
AutoTasker	The action-loop module that re-tunes nodes based on assessments
Baseline	A recorded snapshot of "normal" RF in an area, used for new-vs-known comparison
CE	Circular Error — the radius around a CoT marker representing 1σ position uncertainty
CoC	Center of Control mode — backend aggregates from upstream backends

CoT	Cursor-on-Target — TAK's standard XML-over-UDP situational-awareness wire format
DecisionEngine	The intelligence-layer module that turns tracks + anomalies into threat assessments
DSDFME	The P25 P1+P2 decoder bridge (vendored from DSD-FME)
GPSDO	GPS-Disciplined Oscillator — a GPS receiver that locks the SDR's local oscillator for sub-ppm accuracy
HAE	Height Above Ellipsoid — CoT altitude field; we don't claim it (9999999)
Hit	A persisted detection that the operator (or system) chose to commit to memory
Kujhad	The C++ Predator RF in-band peer protocol (HTTP+JSON, X-Kujhad-Key auth, optional TLS pinning)
MDS	Minimum Detectable Signal — the noise-floor sensitivity in dBm
NF	Noise Figure — front-end noise contribution in dB; lower is better
OTG	USB On-The-Go — host-mode USB on the phone so it can power and command the SDR
PPS	Pulse-Per-Second — the 1 Hz timing pulse from a GPSDO that disciplines the SDR clock
ROE	Rules of Engagement
SAF	Storage Access Framework — Android's file picker; how the app does Import/Export
TAK	Team Awareness Kit — the situational-awareness platform family (ATAK Android, WinTAK Windows, iTAK iOS)
TDOA	Time-Difference-of-Arrival — multi-receiver hyperbolic geolocation
Track	A fused, persistent identity for an emitter — unique by (frequency, modulation, detecting_node_set)
TrackAssociator	The fusion module that decides whether a new event belongs to an existing track or starts a new one
VFO	Variable Frequency Oscillator — in this app, one of two independent tuners

26. Quick reference card

```
START LISTENING        SPEC tab → source dropdown → ▶ Start
DROP A MARKER          tap waterfall peak
NAME A MARKER          tap marker → Assign Marker → tap "Tap to edit"
RECORD BASELINE        BASE → +Add Range → ▶ START RECORDING (5+ min) → Save
USE BASELINE           SYS → Baseline Comparison → load → enable
START AUTO SCAN        MIS → Mission Mode = Scan → +Add Band → SPEC → ▶ Start
ADD KUJHAD PEER        KUJ → Add Peer → Name/Host/Port/Key → Add
PUBLISH ON KUJHAD      KUJ → Listen → port/name/key → toggle Listen
ENABLE ATAK            SYS → ATAK/CoT → server IP+port → Enable CoT
                       (also: COT_REQUIRE_MANUAL_APPROVAL=true in field)
APPROVE A COT          GET /api/v1/approvals → POST /id/approve
START MISSION          MIS → Start mission → name (or POST /api/v1/missions)
END MISSION            MIS → End mission (or POST /api/v1/missions/end)
EXPORT AAR             GET /api/v1/missions/<id>/export

THERMAL ORANGE         reduce sample rate, get out of sun
GPS RED                step outside, wait 60s
KUJHAD RED             check network, ping peer, verify API key
TDOA NO FIX            need ≥2 GPS-synced nodes hearing same emission < 5s apart
HUGE ELLIPSE           low confidence — add more nodes / better timing hardware

BACKEND HEALTH         curl localhost:8000/healthz
FLEET STATUS           curl -H "Authorization: Bearer $TOKEN" .../api/v1/nodes
LIVE TRACKS            curl -H "Authorization: Bearer $TOKEN" .../api/v1/tracks
PENDING APPROVALS      curl -H "Authorization: Bearer $TOKEN" .../api/v1/approvals
LIVE LOGS              journalctl -u predator-rf -f
RESTART BACKEND        sudo systemctl restart predator-rf
BACKUP DB              deploy/backup_mission.sh
```

This document is the contract with the operator. If something here is wrong or out of date, fix it and commit.